

How to Attached Child Tasks to a Parent Task in C#

In this article, I am going to discuss **How to Attached Child Tasks to a Parent Task in C#** with Examples. Please read our previous article where we discussed [Chaining Tasks by Using Continuation Tasks](#) in C# with Examples.

How to Attach Child Tasks to a Parent Task in C#?

A Child Task (or nested task) is a Task that is going to be created by another task called the parent task. A Child Task can be either detached or attached. Now, we need to understand what is detached or attached.

What are Detached or Attached Child Tasks in C#?

A detached child task is a task that executes independently of its parent. In this case, both parent and child are going to be executed independently and the parent will not wait for the Child Task to complete its execution. If any exception is thrown by the child task, then that is not going to be handled by the Parent task. And finally, the Parent Task status does not depend on the Child Task status.

An attached child task is a nested task that is created with [TaskCreationOptions.AttachedToParent](#) option. In this case, the parent will wait for the Child task to complete its execution. If any exception is thrown by the child task, then that is going to be handled by the Parent task. And finally, the Parent task status depends on the Child task status. For a better understanding, please have a look at the following diagram which shows the differences between detached or attached child tasks.

Category	Detached child tasks	Attached child tasks
Parent waits for child tasks to complete.	No	Yes
Parent propagates exceptions thrown by child tasks.	No	Yes
Status of parent depends on status of child.	No	Yes

A task may create any number of attached and detached child tasks. If this is not clear at the moment, don't worry we will try to understand everything in detail with examples.

Important Note: According to MSDN, in most of the scenarios, it is recommended to use detached child tasks, because their relationships with other tasks or parent tasks are less complex. This is the reason why tasks created inside the parent tasks are detached by default, and if you want, you need to explicitly specify [TaskCreationOptions.AttachedToParent](#) option to create an attached child task.

Example to Understand Detached Child Tasks in C#:

Let us understand Detached Child Tasks in C# with an Example. As we already discussed whenever a child task is created by the parent task, then that child task is detached by default. In the below example, the parent task creates one child task. If you run the below example code multiple times, then you may notice that the output is going to vary each time you run the application code. This is because the parent task and child tasks execute independently of each other. The example waits only for the parent task to complete, and the child task may not execute or complete its execution before the console application terminates.

```

using System;
using System.Threading;
using System.Threading.Tasks;
public class Program
{
    public static void Main()
    {
        Console.WriteLine("Main Method Started");
        //Creating the Parent Task
        var parentTask = Task.Factory.StartNew(() => {
            Console.WriteLine("Outer Task Started");
            //Creating the Child Task
            var childTask = Task.Factory.StartNew(() => {
                Console.WriteLine("Child Task Started.");
                Thread.Sleep(5000);
                Console.WriteLine("Child Task Completed");
            });
            Console.WriteLine("Outer Task Completed");
        });
        //Waiting for the Parent Task to completed. Not the Child Task
        parentTask.Wait();
        Console.WriteLine("Main Method Completed");
        Console.ReadKey();
    }
}

```

Output: As you can see in the below output, the main task is not waiting for the child task to complete its execution.

```

Main Method Started
Outer Task Started
Outer Task Completed
Child Task Started.
Main Method Completed
Child Task Completed

```

How Parent task will wait for the detached task to complete its execution?

If the child task is represented by **Task<TResult>** object rather than a Task object, then you can ensure that the parent task will wait for the child task to complete its execution by accessing the **Task<TResult>.Result** property of the child task even though it is a detached child task. The reason for this is the Result property will block the parent task until the child task completes its execution. For a better understanding, please have a look at the below example.

```

using System;
using System.Threading;
using System.Threading.Tasks;
public class Program
{
    public static void Main()

```

```

{
    Console.WriteLine("Main Method Started");
    //Creating the Parent Task
    var parentTask = Task<string>.Factory.StartNew(() => {
        Console.WriteLine("Outer Task Started");
        //Creating the Child Task
        var childTask = Task<string>.Factory.StartNew(() => {
            Console.WriteLine("Child Task Started");
            Thread.Sleep(5000);
            Console.WriteLine("Child Task Completed");
            return "Task Completed";
        });

        // Parent Task will wait for detached Child Task to complete its execution
        return childTask.Result;
    });
    //Here, parentTask.Result will block the Main thread, till the Parent Task
    complete its execution
    Console.WriteLine($"Parent Task Returned: {parentTask.Result}");
    Console.WriteLine("Main Method Completed");
    Console.ReadKey();
}
}

```

Output:

```

Main Method Started
Outer Task Started
Child Task Started
Child Task Completed
Parent Task Returned: Task Completed
Main Method Completed

```

Example to Understand Attached Child Tasks in C#:

As we already discussed the attached child tasks are closely synchronized with the parent task i.e. the task that created it. We also discussed that by default the Child task is going to be detached. If you want to create an attached child task, then you need to use [TaskCreationOptions.AttachedToParent](#) option in the task creation statement. For a better understanding, please have a look at the following example. In the below code, the attached child task completes before its parent. As a result, the output of this example is going to be the same each time you run the code.

```

using System;
using System.Threading;
using System.Threading.Tasks;
public class Program
{
    public static void Main()
    {

```

```

        Console.WriteLine("Main Method Started");
        //Creating the Parent Task
        var parentTask = Task.Factory.StartNew(() => {
            Console.WriteLine("Outer Task Started");
            //Creating the Child Task
            var childTask = Task.Factory.StartNew(() => {
                Console.WriteLine("Child Task Started");
                Thread.Sleep(5000);
                Console.WriteLine("Child Task Completed");
            }, TaskCreationOptions.AttachedToParent);
            Console.WriteLine("Outer Task Completed");
        });
        //Waiting for the Parent Task to completed.
        parentTask.Wait();
        Console.WriteLine("Main Method Completed");
        Console.ReadKey();
    }
}

```

Output:

```

Main Method Started
Outer Task Started
Outer Task Completed
Child Task Started
Child Task Completed
Main Method Completed

```

How to Prevent Parent tasks from attached Child Tasks in C#?

The point that you need to remember is that a Child task can only attach to its parent if its parent does not prohibit attached child tasks. Parent tasks can explicitly prevent child tasks from attaching to them by specifying the `TaskCreationOptions.DenyChildAttach` option. Let us understand this with an example. In the below example, we created the Parent Task with `TaskCreationOptions.DenyChildAttach` option and created the Child task with `TaskCreationOptions.AttachedToParent` option. Even though we attached the child task, it is not going to work.

```

using System;
using System.Threading;
using System.Threading.Tasks;
public class Program
{
    public static void Main()
    {
        Console.WriteLine("Main Method Started");
        //Creating the Parent Task with DenyChildAttach Option
        var parentTask = Task.Factory.StartNew(() => {
            Console.WriteLine("Outer Task Started");
            //Creating the Child Task with AttachedToParent
            var childTask = Task.Factory.StartNew(() => {

```

```

        Console.WriteLine("Child Task Started");
        Thread.Sleep(5000);
        Console.WriteLine("Child Task Completed");
    }, TaskCreationOptions.AttachedToParent);
    Console.WriteLine("Outer Task Completed");
}, TaskCreationOptions.DenyChildAttach);
//Waiting for the Parent Task to completed.
parentTask.Wait();
Console.WriteLine("Main Method Completed");
Console.ReadKey();
}
}

```

Output:

```

Main Method Started
Outer Task Started
Outer Task Completed
Child Task Started
Main Method Completed
Child Task Completed

```

The Parent tasks implicitly prevent child tasks from attaching to them if they are created by calling the **Task.Run** method. For a better understanding, please have a look at the following example. Here, we are creating the Parent Task using the Task.Run method.

```

using System;
using System.Threading;
using System.Threading.Tasks;
public class Program
{
    public static void Main()
    {
        Console.WriteLine("Main Method Started");
        //Creating the Parent Task using Task.Run Method
        var parentTask = Task.Run(() => {
            Console.WriteLine("Outer Task Started");
            //Creating the Child Task with AttachedToParent
            var childTask = Task.Factory.StartNew(() => {
                Console.WriteLine("Child Task Started");
                Thread.Sleep(5000);
                Console.WriteLine("Child Task Completed");
            }, TaskCreationOptions.AttachedToParent);
            Console.WriteLine("Outer Task Completed");
        });
        //Waiting for the Parent Task to completed.
        parentTask.Wait();
        Console.WriteLine("Main Method Completed");
        Console.ReadKey();
    }
}

```

```
}
```

Output:

```
Main Method Started
Outer Task Started
Outer Task Completed
Child Task Started
Main Method Completed
Child Task Completed
```

Exceptions in Detached Child Task in C#

If a detached child task throws an exception, then that exception is not going to be propagated to the Parent Task. For a better understanding, please have a look at the following example. Here, the child task is created without using [TaskCreationOptions.AttachedToParent](#) option and within the child task we are throwing an exception i.e. Divide By Zero Exception and you will see that the exception is not going to be handled by the parent task.

```
using System;
using System.Threading.Tasks;
public class Program
{
    public static void Main()
    {
        Console.WriteLine("Main Method Started");
        try
        {
            //Creating the Parent Task using Task.Run Method
            var parentTask = Task.Factory.StartNew(() =>
            {
                Console.WriteLine("Outer Task Started");
                //Creating the Child Task with AttachedToParent
                var childTask = Task.Factory.StartNew(() =>
                {
                    Console.WriteLine("Child Task Started");
                    int x = 10, y = 0;
                    int z = x / y; //It will throw an Exception
                    Console.WriteLine("Child Task Completed");
                });
                Console.WriteLine("Outer Task Completed");
            });
            //Waiting for the Parent Task to completed.
            parentTask.Wait();
        }
        catch (Exception ex)
        {
            Console.WriteLine($"Exception Occurred: {ex.Message}");
        }
    }
}
```

```

        Console.WriteLine("Main Method Completed");
        Console.ReadKey();
    }
}

```

Output:

```

Main Method Started
Outer Task Started
Outer Task Completed
Child Task Started
Main Method Completed

```

Exceptions in Attached Child Task in C#

If an attached child task throws an exception, then that exception is automatically propagated to the parent task and back to the thread that waits or tries to access the task's `Task<TResult>.Result` property. Therefore, by using attached child tasks, you can handle all exceptions at just one point. For a better understanding, please have a look at the below example. Here, the child task is created using `TaskCreationOptions.AttachedToParent` option and within the child task we are throwing an exception i.e. Divide By Zero Exception and you will see that the exception is going to be handled by the parent task.

```

using System;
using System.Threading.Tasks;
public class Program
{
    public static void Main()
    {
        Console.WriteLine("Main Method Started");
        try
        {
            //Creating the Parent Task using Task.Run Method
            var parentTask = Task.Factory.StartNew(() =>
            {
                Console.WriteLine("Outer Task Started");
                //Creating the Child Task with AttachedToParent
                var childTask = Task.Factory.StartNew(() =>
                {
                    Console.WriteLine("Child Task Started");
                    int x = 10, y = 0;
                    int z = x / y; //It will throw an Exception
                    Console.WriteLine("Child Task Completed");
                }, TaskCreationOptions.AttachedToParent);
                Console.WriteLine("Outer Task Completed");
            });
            //Waiting for the Parent Task to completed.
            parentTask.Wait();
        }
        catch (Exception ex)

```

```
    {  
        Console.WriteLine($"Exception Occurred: {ex.Message}");  
    }  
    Console.WriteLine("Main Method Completed");  
    Console.ReadKey();  
}  
}
```

Output:

```
Main Method Started  
Outer Task Started  
Outer Task Completed  
Child Task Started  
Exception Occurred: One or more errors occurred.  
Main Method Completed
```

In the next article, I am going to discuss [ValueTask in C#](#) with Examples. In this article, I try to explain **How to Attached Child Tasks to a Parent Task in C#** with Examples. I hope you enjoy this How to Attach Child Tasks to a Parent Task in C# with Examples article.